

MVLU COLLEGE

AI PRACTICAL NO. 6

AIM:Feed Forward Backpropagation Neural Network

- Implement the Feed Forward Backpropagation algorithm to train a neural network.
- Use a given dataset to train the neural network for a specific task.
- Evaluate the performance of the trained network on test data.

#CODE:

```
# PRACTICAL 6: FEED FORWARD BACKPROPAGATION NEURAL NETWORK
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
# Upload dataset
uploaded = files.upload()
# Read dataset
df = pd.read_csv("Students Performance.csv")
print("First 5 rows:")
print(df.head())
print("\nDataset Shape:")
print(df.shape)
print("\nDataset Information:")
print(df.info())
# Find median Placement Score
placement_median = df["Placement_Score"].median()
print("\nMedian Placement Score:")
print(placement_median)
# Create target class
# 1 = High Placement Score
# 0 = Low Placement Score

df["Placement_Class"] = (
    df["Placement_Score"] >= placement_median
).astype(int)
# Features
X = df.drop(
    columns=[
```

MVLU COLLEGE

AI PRACTICAL NO. 6

```
        "Placement_Score",
        "Placement_Class"    ]
)
# Target
y = df["Placement_Class"]
print("\nClass Distribution:")
print(y.value_counts())
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
# Standardize data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Create Feed Forward Neural Network
model = MLPClassifier(
    hidden_layer_sizes=(16, 8),
    activation="relu",
    solver="adam",
    learning_rate_init=0.001,
    max_iter=1500,
    random_state=42
)
# Train using Backpropagation
model.fit(X_train_scaled, y_train)
# Predictions
y_pred = model.predict(X_test_scaled)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nNeural Network Accuracy:")
print(accuracy)
print("\nAccuracy Percentage:")
print(round(accuracy * 100, 2), "%")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

MVLU COLLEGE

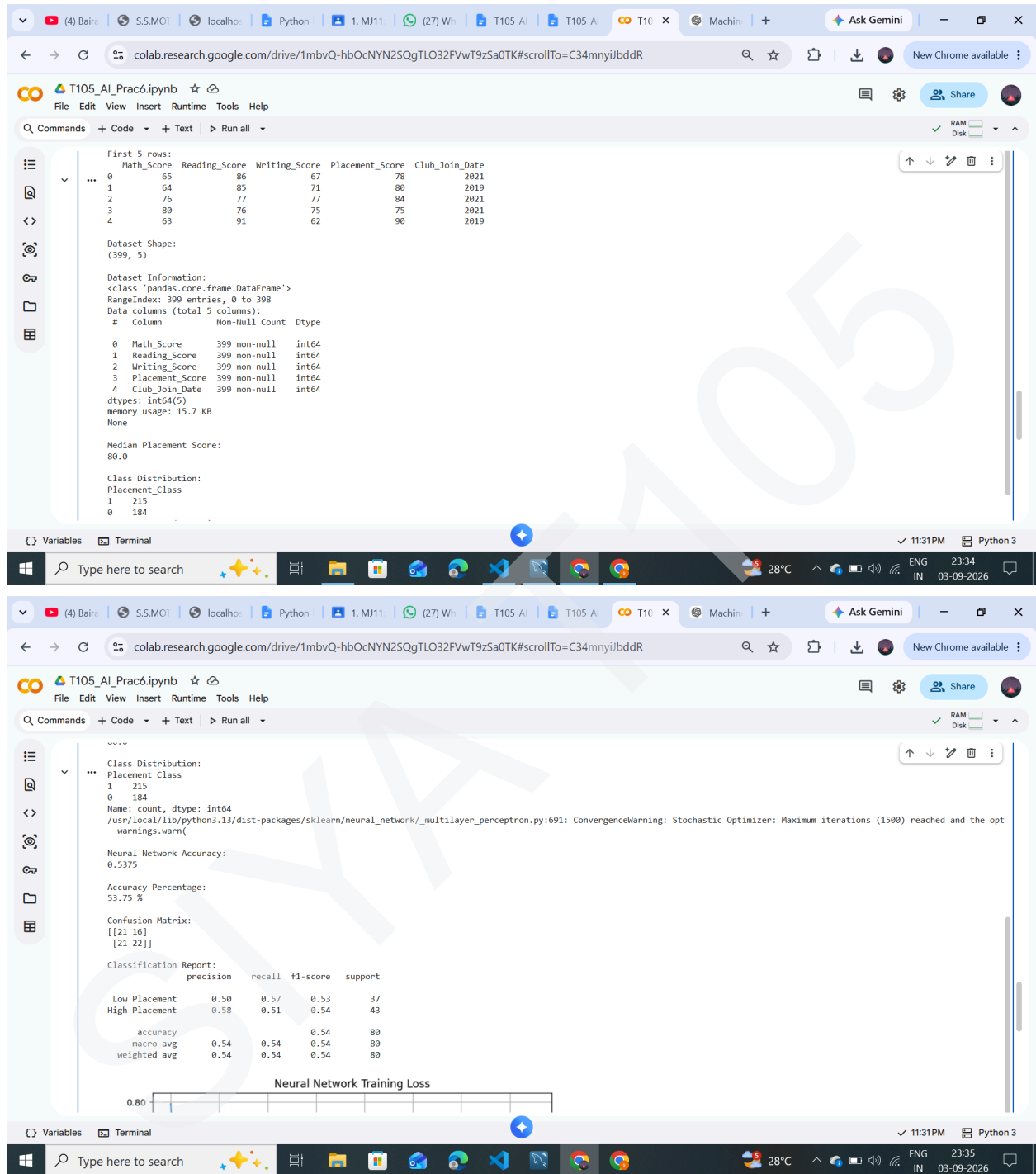
AI PRACTICAL NO. 6

```
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=[
            "Low Placement",
            "High Placement"
        ]
    )
)
# Plot training loss
plt.figure(figsize=(8, 5))
plt.plot(model.loss_curve_)
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("Neural Network Training Loss")
plt.grid()
plt.show()
# Actual vs Predicted
result = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})
print("\nActual vs Predicted:")
print(result.head(15))
```

MVLU COLLEGE

AI PRACTICAL NO. 6

#OUTPUT:



MVLU COLLEGE

AI PRACTICAL NO. 6

Neural Network Training Loss

